

SafeMerchant Risk Agent

Autonomous AI Risk Manager — End-to-End Backend Architecture & Flow Specification

1. System Purpose & Core Invariants

SafeMerchant is a production-grade, **defense-only agentic risk management system** engineered to autonomously evaluate, evidence, and resolve payment disputes and chargebacks. Built specifically for the Razorpay Buildathon (Track 2), the backend combines high-speed webhook ingestion with stateful LangGraph orchestration, deterministic zero-hallucination evidence verification, Human-in-the-Loop (HITL) checkpoints, asynchronous ReportLab PDF generation, dual cloud storage, and an independent transactional metrics & audit layer.

DEFENSE-ONLY SAFETY INVARIANT

The system is architecturally constrained from performing offensive or unverified mutations. All internal database access against merchant tables (orders, shipping, customer chat transcripts, risk telemetry) is strictly **READ-ONLY**. The only external write operation permitted is the Razorpay refund API call — and only when customer legitimacy is verified and disputed amounts remain below the configured automated safety threshold (₹10,000).

2. End-to-End Operational Pipeline & Flow

Step 1: Webhook Ingestion	FastAPI receives Razorpay 'payment.dispute.created' event. Verbatim payload is committed to immutable 'dispute_events', a record in 'disputes' is created with status 'processing', and 'dispute_received' is broadcast to connected WebSocket dashboards within 35ms.
Step 2: Evidence Retrieval	Super Step 1 executes read-only queries across orders, shipping logs, customer communications transcripts, and 2FA/IP risk telemetry via EvidenceRepository.
Step 3: Triage & Winnability Scoring	Super Step 2 evaluates evidence completeness (delivery proof +25%, signed-by +10%, 2FA +10%) and runs an independent customer legitimacy check to identify cases where the customer is genuinely entitled to a refund.
Step 4: Grounded Drafting & Verification	Super Step 3 executes a 3-stage LLM pipeline: Step A extracts structured facts from communications; Step B generates a structured draft (<=1000 chars); Step C deterministically verifies every claim against dotted database paths, dropping ungrounded statements.
Step 5: Conditional Gate Edge	Evaluates winnability score, amount ceiling, and legitimacy flag. Routes dispute to 'auto_submit', 'human_review', 'accept_loss', 'auto_refund', or 'refund_review'.
Step 6: Stateful HITL Breakpoint	High-value (>₹10k) or borderline cases pause at LangGraph 'interrupt_before' breakpoints. State is persisted in PostgreSQL via AsyncPostgresSaver until a reviewer approves via REST API.
Step 7: Async PDF Generation & Dual Upload	EvidenceWorkerPool (capped at 5 concurrent tasks) generates multi-page ReportLab PDFs in memory, uploads to Razorpay Documents API and Supabase Storage, and contests the dispute via Razorpay Disputes API.
Step 8: Transactional Metrics & WebSocket Refresh	Final status and outcomes are committed to PostgreSQL, updating dispute_metrics_daily and dispute_breakdowns. A lightweight 'metrics_stale' signal notifies UI to refresh.

3. Detailed Module & Subsystem Architecture

Component / Subsystem	Location	Responsibilities & Key Mechanics
FastAPI Application & Lifespan	app/main.py	Initializes AsyncEngine, compiles LangGraph state graph with AsyncPostgresSaver checkpointer context, ensures Supabase bucket existence, and starts the background EvidenceWorkerPool on startup.
LangGraph State Machine	app/dispute/agent/	StateGraph implementation with TypedDict state containing 35+ fields across 4 execution stages. Connects 3 super steps, conditional gate routing, and interrupt breakpoints before HITL nodes.
Deterministic Grounding Verifier	app/dispute/agent/nodes/_verification.py	Walks every factual claim in LLM outputs, resolves dotted source paths (e.g. 'shipping.delivery_status'), drops ungrounded statements, and reconstructs verified explanation letters.
Evidence Repository Layer	app/dispute/repository.py	Async read-only SQLAlchemy repository providing strict isolation from merchant production records. Eagerly loads multi-table evidence packages.
Dispute Lifecycle Repository	app/dispute/dispute_repository.py	Manages state transitions, status updates, evidence job enqueueing, and transactional human edits with atomic audit logging.
Evidence Job Worker Pool	app/dispute/worker.py	Background queue worker with asyncio.Semaphore(5) and PostgreSQL row-level locking (SELECT ... FOR UPDATE SKIP LOCKED) to prevent race conditions.
Chargeback PDF Renderer	app/proof_renderer/	ReportLab rendering engine generating bank-ready PDFs in memory (delivery proofs, address mismatch warnings, OTP logs, carrier timeline, fraud signals, chat transcripts).
Supabase Storage Client	app/core/storage.py	Async HTTPX client managing bucket creation, in-memory PDF uploads, and generating short-lived 1-hour signed URLs for direct client preview.
Outbound Submission Pipeline	app/dispute/submission.py	Coordinates dual-upload to Razorpay Documents API and Supabase Storage, dispatches contest requests, and gracefully classifies sandbox limitations.
Metrics & Transactional Audit	app/dispute/metrics_repository.py	Separated OLAP layer tracking daily summaries, dimensional breakdowns (reason_code, outcome, phase), repeat customer patterns, and human edit audit trails.

4. Defense-Only Gate Decision Matrix

The conditional edge function (gate_decision in app/dispute/agent/nodes/gate.py) routes the workflow based on winnability confidence, financial thresholds, and consumer legitimacy signals:

Condition / Signal	Amount Limit	Recommended	Gate Action	Operational Outcome
Customer Legitimacy Flag = TRUE	<= INR 10,000	refund_customer	auto_refund	Executes Razorpay Refund API automatically. Customer gets money back without manual friction.
Customer Legitimacy Flag = TRUE	> INR 10,000	refund_customer	refund_review	HITL interrupt breakpoint. Requires human reviewer approval prior to high-value refund.
Winnability Score >= 85%	<= INR 10,000	contest	auto_submit	Auto-enqueues asynchronous evidence generation job. PDF rendered, uploaded, and contest submitted.
Winnability Score < 85% OR Amount > 10k	Any	contest	human_review	HITL interrupt breakpoint. State persisted in PostgreSQL checkpointer awaiting human review.
Winnability Score < 30% (No Legitimacy)	Any	accept_loss	accept_loss	Resolves dispute as accepted loss to avoid wasting merchant dispute fees on unwinnable claims.

5. Relational Data Model & Storage Schema

Table Name	Access Type	Description & Contents
orders	Read-Only	Core merchant ledger (order_id, payment_id, customer_email, amount_inr, created_at).
shipping_logs	Read-Only	Physical logistics evidence (tracking_id, courier_partner, delivery_status, signed_by, delivery_timestamp).
customer_communications	Read-Only	Customer interaction transcripts (ticket_id, channel, message_transcript, logged_at).
risk_signals	Read-Only	Network & authentication telemetry (signal_id, ip_address, device_fingerprint, is_2fa_verified, account_age_days).
disputes	Read/Write	Operational lifecycle record (id, status, amount_paise, reason_code, document_id, storage_path, phase, outcome, history JSONB).
evidence_jobs	Queue	Asynchronous PDF rendering and upload queue (id, dispute_id, status, attempts, error_message).
dispute_events	Append-Only	Immutable raw webhook log storing every incoming JSON payload verbatim with occurred_at timestamp.
dispute_audit_log	Append-Only	Transactional log recording every manual field modification made via PATCH /disputes/{id}.
dispute_submission_log	Append-Only	Detailed log of outbound Razorpay document upload and contest API calls and response payloads.
dispute_metrics_daily	Aggregated	Pre-aggregated daily metrics (date, total_disputes, won, lost, action_required, amounts won/lost/at-risk, sla_breached).
dispute_breakdowns	Materialized	Pre-aggregated dimensional counts by reason_code, outcome, and phase.
checkpoints (LangGraph)	Managed	PostgreSQL state persistence tables (checkpoints, checkpoint_blobs, checkpoint_writes) for thread state.

6. REST API & Real-Time WebSocket Specification

Method	Endpoint Path	Description
POST	/api/v1/webhook	Razorpay dispute webhook ingestion (Returns HTTP 202 immediately).
GET	/api/v1/disputes	List historical disputes with latest evidence job status.
GET	/api/v1/disputes/{id}	Get detailed state and evidence job status for a single dispute.
POST	/api/v1/disputes/{id}/review	Submit human review decision (accept/reject) and resume graph.
POST	/api/v1/disputes/{id}/retry-evidence	Manually retry a failed evidence generation job.
GET	/api/v1/disputes/{id}/evidence-url	Generate short-lived signed URL for Supabase Storage PDF.
GET	/api/v1/metrics/summary	Query aggregated daily totals across a date range.
GET	/api/v1/metrics/breakdown	Query pre-aggregated counts by reason_code, outcome, or phase.
GET	/api/v1/metrics/repeat-patterns	Find repeat customer emails appearing across multiple disputes.
PATCH	/api/v1/disputes/{id}	Transactional human edit with atomic dispute_audit_log write.
GET	/api/v1/disputes/{id}/audit	Fetch complete audit trail for a dispute.
WS	/ws/dashboard	Real-time WebSocket stream for node progress and metrics_state invalidations.